1. Student Performance Trend Analysis:

1. Problem Statement

To develop a Python program that analyzes student performance across three examinations. The program compares marks, calculates improvements and declines, identifies consistently weak subjects, and classifies students according to their average marks.

2. Objectives

- To compare student marks across three examinations.
- To calculate improvements and declines in marks.
- To identify subjects in which the student consistently scores below 40.
- To calculate the student's overall average.
- To classify the student based on the average marks.

3. Assumptions

1. The student studies Python, Maths, and English.
2. Each subject has three examination scores.
3. Each examination is out of 100 marks.
4. A mark below 40 in all three examinations indicates a consistently weak subject.
5. Student classification is based on the overall average.

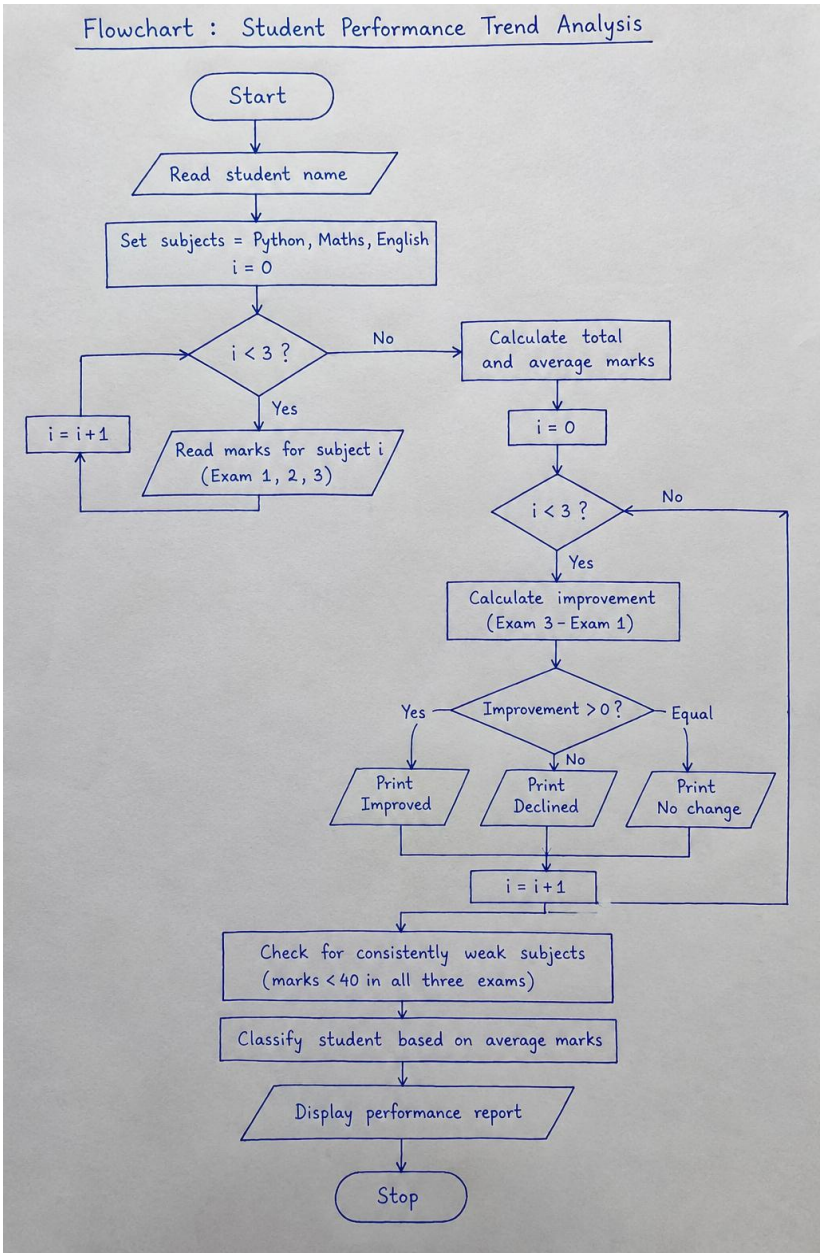
Average marks	Classification
80–100	Excellent
60–79.99	Good
40–59.99	Average

Below 40	Needs Improvement

4. Algorithm

1. Start the program.
2. Read the student's name.
3. Define the three subjects.
4. Input marks for all subjects in three examinations.
5. Calculate the total and average marks.
6. Compare the first and third examination marks for each subject.
7. Display whether performance improved, declined, or remained unchanged.
8. Identify subjects with marks below 40 in all three examinations.
9. Classify the student based on the average marks.
10. Display the performance report.
11. Stop the program.

5. Flowchart



6. Pseudocode

```
BEGIN
    INPUT student name
    SET subjects = Python, Maths, English

    FOR each subject
```

```
        INPUT Exam 1 marks
        INPUT Exam 2 marks
        INPUT Exam 3 marks
    END FOR

    CALCULATE total marks
    CALCULATE average marks

    FOR each subject
        CALCULATE improvement = Exam 3 - Exam 1

        IF improvement > 0
            DISPLAY Improved
        ELSE IF improvement < 0
            DISPLAY Declined
        ELSE
            DISPLAY No change
        END IF

        IF all three marks are below 40
            DISPLAY Weak subject
        END IF
    END FOR

    IF average >= 80
        DISPLAY Excellent
    ELSE IF average >= 60
        DISPLAY Good
    ELSE IF average >= 40
        DISPLAY Average
    ELSE
        DISPLAY Needs Improvement
    END IF
END
```

7. Python Source Code

```
# Student Performance Trend Analysis
name = input("Enter student name: ")

subjects = ["Python", "Maths", "English"]

exam1 = [ ]
exam2 = [ ]
exam3 = [ ]

# Input marks for three examinations
for subject in subjects:
    print("\nEnter marks for", subject)
    m1 = float(input("Exam 1 marks: "))
    m2 = float(input("Exam 2 marks: "))
    m3 = float(input("Exam 3 marks: "))

    exam1.append(m1)
    exam2.append(m2)
    exam3.append(m3)

# Calculate total and average marks
total = sum(exam1) + sum(exam2) + sum(exam3)
```

```
average = total / 9


print("\n--- Student Performance Report ---")
print("Student Name:", name)


# Compare examination marks
for i in range(len(subjects)):
    improvement = exam3[i] - exam1[i]


    print("\nSubject:", subjects[i])
    print("Exam 1:", exam1[i])
    print("Exam 2:", exam2[i])
    print("Exam 3:", exam3[i])


    if improvement > 0:
        print("Performance: Improved by", improvement, "marks")
    elif improvement < 0:
        print("Performance: Declined by", abs(improvement), "marks")
    else:
        print("Performance: No change")


# Identify consistently weak subjects
```

```
print("\n--- Weak Subjects ---")

weak_found = False


for i in range(len(subjects)):

    if exam1[i] < 40 and exam2[i] < 40 and exam3[i] < 40:

        print(subjects[i], "is consistently weak")

        weak_found = True


if not weak_found:

    print("No consistently weak subjects")


# Classify the student

print("\nAverage Marks:", round(average, 2))


if average >= 80:

    print("Classification: Excellent")

elif average >= 60:

    print("Classification: Good")

elif average >= 40:

    print("Classification: Average")

else:

    print("Classification: Needs Improvement")
```

8. Test Cases

Test case	Input condition	Expected result
1	Average = 90	Excellent
2	Average = 70	Good
3	Average = 50	Average
4	Average = 30	Needs Improvement
5	All three Maths marks are below 40	Maths identified as weak
6	Exam 3 marks exceed Exam 1 marks	Improved performance
7	Exam 3 marks are lower than Exam 1 marks	Declined performance
8	Exam 3 marks equal Exam 1 marks	No change

9. Output Screenshot



10. Conclusion

The Student Performance Trend Analysis program successfully compares marks across three examinations. It calculates average marks, identifies improvements and declines, detects consistently weak subjects, and classifies students according to their performance. This program demonstrates the use of Python variables, lists, loops, conditional statements, and built-in functions.

2.Student Timetable Conflict Detection:

1. Problem Statement

To develop a Python program to detect timetable conflicts by comparing course timings and room numbers. The program identifies courses scheduled at the same time in the same room.

2. Objectives

- To store course details using tuples.
- To compare course timings.
- To identify timetable conflicts.
- To check room availability.
- To validate the timetable.

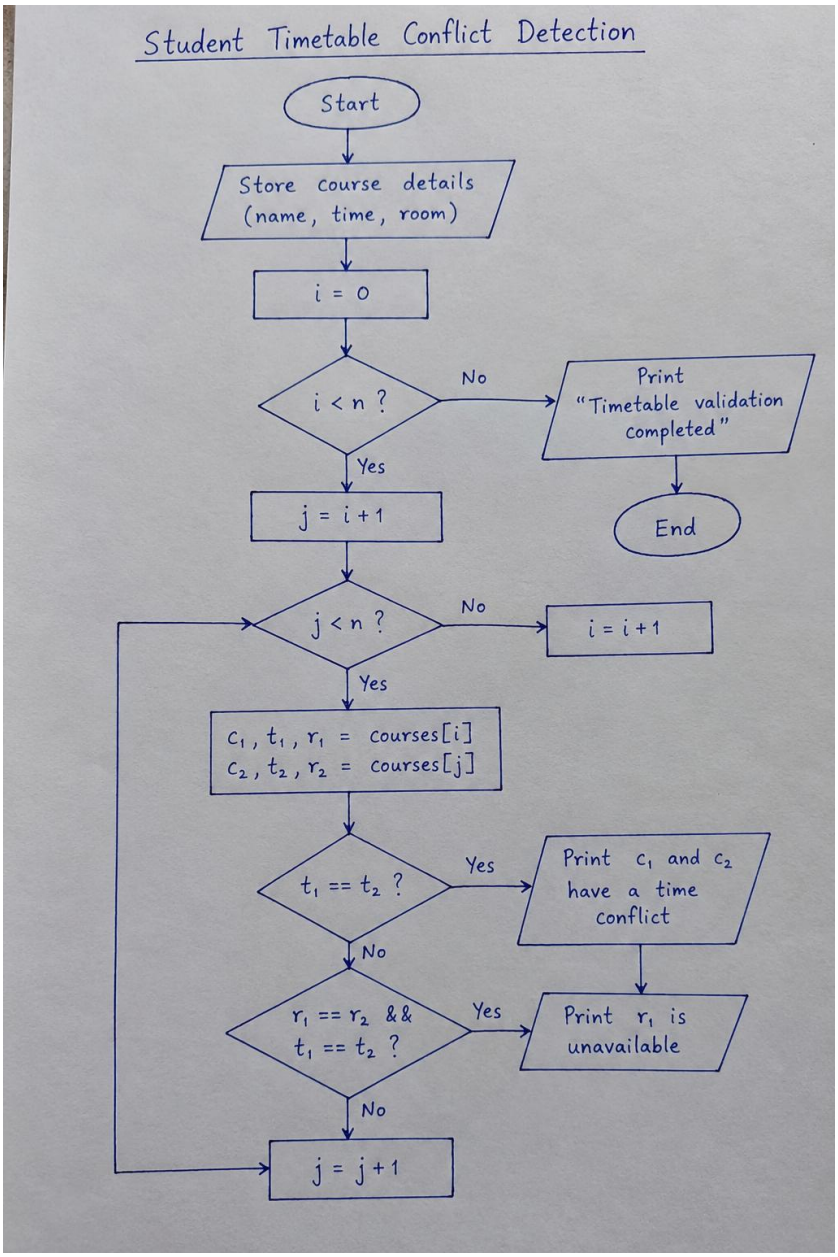
3. Assumptions

- Each course has a name, time slot, and room number.
- Time slots are given as strings, such as `"9-10"`.
- Two courses conflict if they have the same time slot.
- A room is unavailable if two courses use the same room at the same time.

4. Algorithm

1. Start the program.
2. Store course details in a list of tuples.
3. Compare each course with the remaining courses.
4. Check whether their time slots are equal.
5. If equal, display a time conflict.
6. Check whether both the time and room are equal.
7. If equal, display that the room is unavailable.
8. Display the timetable validation message.
9. Stop the program.

5. Flowchart



6. Pseudocode

```
START
Store course details
```

```
FOR each pair of courses
    Compare their time slots
    IF time slots are equal
        Display time conflict
    END IF

    IF room numbers and time slots are equal
        Display room unavailable
    END IF
END FOR

Display timetable validation completed
STOP
```

7. Python Source Code

```
courses = [
  ("Python", "9-10", "Room1"),
  ("Maths", "10-11", "Room2"),
  ("English", "9-10", "Room1")
]


for i in range(len(courses)):
  for j in range(i + 1, len(courses)):
    c1, t1, r1 = courses[i]
    c2, t2, r2 = courses[j]


    if t1 == t2:
      print(c1, "and", c2, "have a time conflict")


    if r1 == r2 and t1 == t2:
```

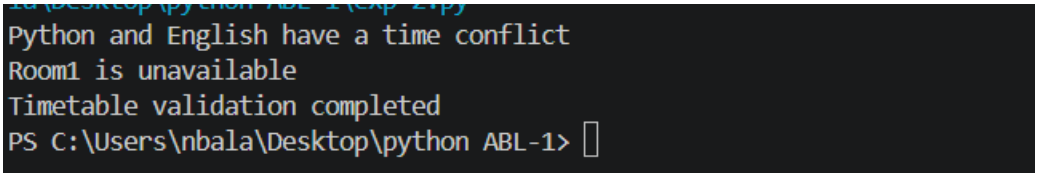
```
        print(r1, "is unavailable")

print("Timetable validation completed")
```

8. Test Cases

Test case	Courses compared	Expected result
1	Python and Maths	No conflict
2	Python and English	Time conflict
3	Python and English in Room1	Room unavailable
4	Complete timetable	Validation message displayed

9.Screenshot Output



10. Conclusion

The program compares course schedules and identifies time conflicts and room clashes. It demonstrates Python lists, tuples, loops, conditional statements, and string comparison. It helps students understand the basic process of timetable validation.

3.Hospital Ward Occupancy Monitoring:

1. Problem statement

Hospitals need to monitor the number of occupied beds in each ward every day. This Python program analyzes bed occupancy counts, calculates bed utilization percentages, identifies capacity violations, and finds the peak-demand period.

2. Objectives

- To store ward occupancy counts using nested lists.
- To calculate bed utilization percentages.
- To identify wards that exceed their bed capacity.
- To find the day and ward with the highest occupancy.
- To display a clear occupancy report.

3. Assumptions

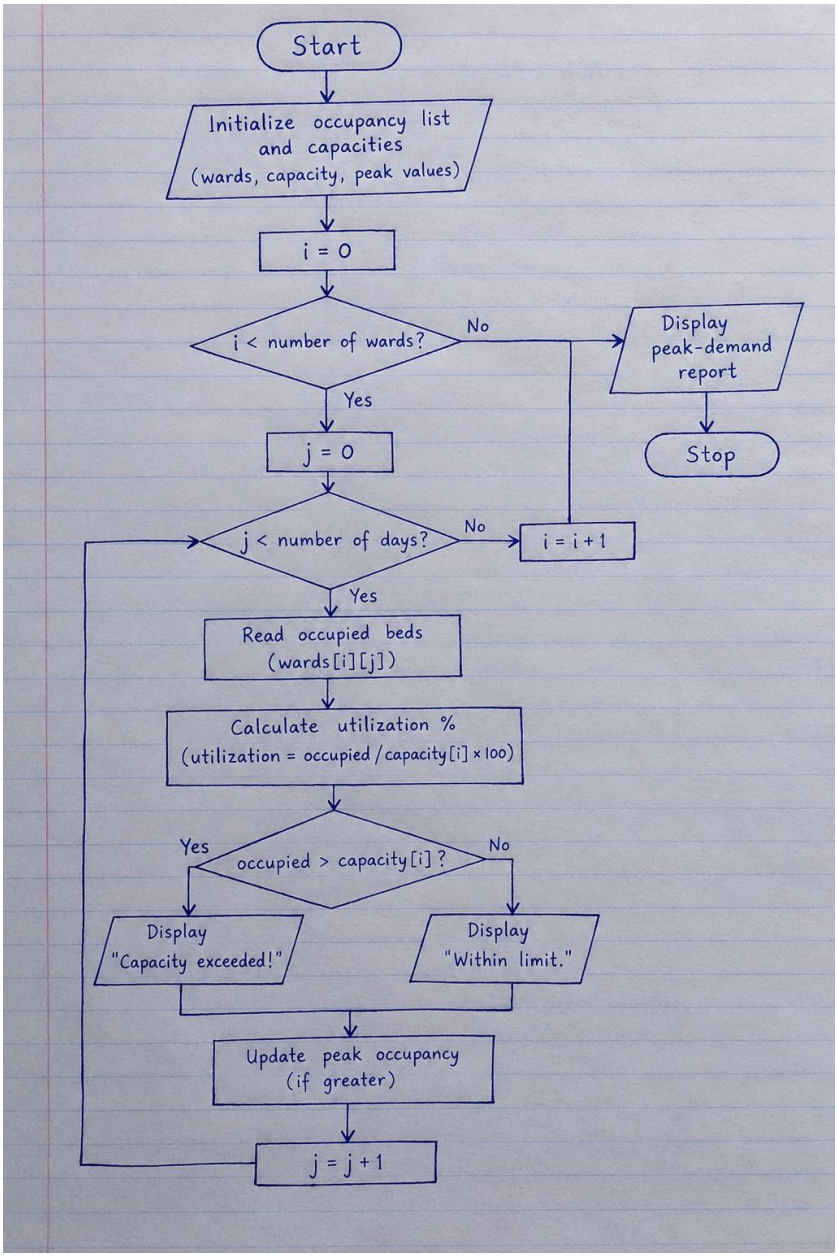
1. The hospital has three wards.
2. Each ward is monitored for three days.
3. Each ward has a fixed bed capacity.
4. Occupancy counts are non-negative integers.
5. Occupancy above capacity is considered a capacity violation.

4. Algorithm

1. Start.
2. Initialize the nested list of occupancy counts.
3. Store the bed capacity of each ward.
4. Initialize the peak occupancy variables.
5. Use an outer loop to select each ward.
6. Use an inner loop to process each day's occupancy.
7. Calculate the utilization percentage.
8. Check whether occupancy exceeds capacity.
9. Update the peak-demand details when a higher occupancy is found.
10. Display the peak-demand report.

11. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
    Store occupancy counts in a nested list
    Store capacity of each ward
```

```
    Set peak occupancy to 0

    FOR each ward
        FOR each day
            Get occupied beds
            Calculate utilization percentage
            Display occupancy and utilization

            IF occupied beds > capacity
                Display capacity warning
            ELSE
                Display within-limit message
            END IF

            IF occupied beds > peak occupancy
                Update peak occupancy, ward, and day
            END IF
        END FOR
    END FOR

    Display peak-demand report
END
```

7. Python source code

```
# Hospital Ward Occupancy Monitoring


# Bed occupancy counts for each ward over 3 days

wards = [

  [8, 10, 12],  # Ward 1

  [15, 18, 20], # Ward 2
```

```
    [5, 7, 9]    # Ward 3
]

# Total bed capacity of each ward
capacity = [10, 15, 18]

peak_occupancy = 0
peak_ward = 0
peak_day = 0

for i in range(len(wards)):
    print("\nWard", i + 1)

    for j in range(len(wards[i])):
        occupied = wards[i][j]

        # Calculate utilization percentage
        utilization = (occupied / capacity[i]) * 100

        print("Day", j + 1)
        print("Occupied beds:", occupied)
        print("Utilization:", round(utilization, 2), "%")
```

```
        # Detect capacity violations
        if occupied > capacity[i]:
            print("Warning: Capacity exceeded!")
        else:
            print("Capacity is within limit.")

        # Find peak-demand period
        if occupied > peak_occupancy:
            peak_occupancy = occupied
            peak_ward = i + 1
            peak_day = j + 1

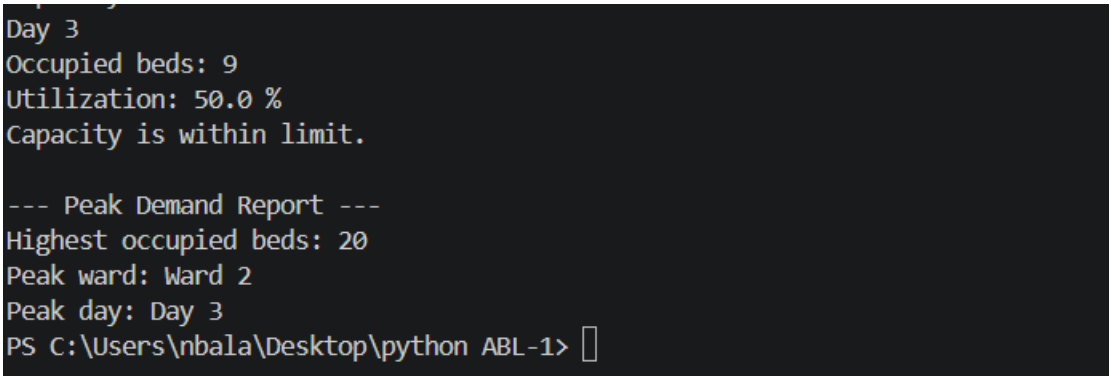
print("\n--- Peak Demand Report ---")
print("Highest occupied beds:", peak_occupancy)
print("Peak ward: Ward", peak_ward)
print("Peak day: Day", peak_day)
```

8. Test cases

Test case	Ward capacity	Occupied beds	Expected result

1	10	8	80% utilization; within limit
2	10	10	100% utilization; within limit
3	10	12	120% utilization; capacity exceeded
4	15	20	133.33% utilization; capacity exceeded
5	18	9	50% utilization; within limit

9. Output screenshot



10. Conclusion

The Hospital Ward Occupancy Monitoring program successfully calculates bed utilization percentages, detects capacity violations, and identifies peak-demand periods using Python nested lists, loops, arithmetic operations, and conditional statements.

4.University Scholarship Allocation:

1. Problem Statement

To develop a Python program that evaluates students for university scholarships based on aggregate marks, subject-wise minimum marks, attendance percentage, and family income. The program should identify eligible students, rank them according to merit, and resolve ties using predefined rules.

2. Objectives

- To check scholarship eligibility using specified conditions.
- To verify minimum marks in each subject.
- To check attendance and family income limits.
- To rank eligible students based on aggregate marks.
- To resolve ties using attendance and family income.

3. Assumptions

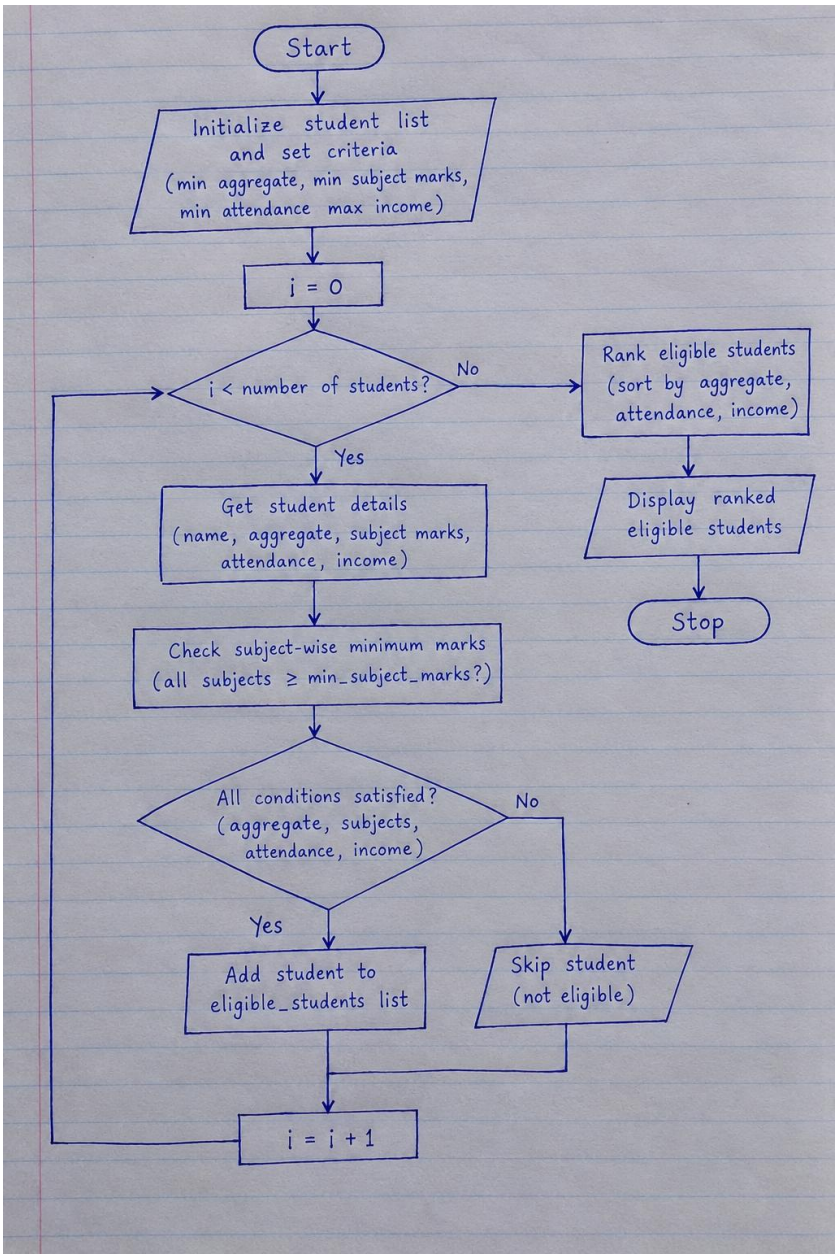
1. Minimum aggregate marks required: 75.
2. Minimum marks required in each subject: 60.
3. Minimum attendance required: 75%.
4. Maximum annual family income: ₹2,00,000.
5. Higher aggregate marks receive a higher rank.
6. If aggregate marks are equal, higher attendance gets priority.
7. If aggregate marks and attendance are equal, lower family income gets priority.

4. Algorithm

1. Start.
2. Store student details in a nested list.
3. Set the scholarship eligibility criteria.
4. Create an empty list for eligible students.
5. Read each student's aggregate marks, subject marks, attendance, and family income.
6. Check whether all subject marks meet the minimum requirement.

7. Check all eligibility conditions.
8. Add eligible students to the eligible list.
9. Sort students according to the tie-breaking rules.
10. Display the ranked list of eligible students.
11. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN

Store student details
Set minimum aggregate marks
Set minimum subject marks
Set minimum attendance
Set maximum family income

Create empty eligible_students list

FOR each student
    Check subject-wise minimum marks

    IF aggregate marks meet the minimum
       AND all subject marks meet the minimum
       AND attendance meets the minimum
       AND family income is within the limit

        Add student to eligible_students
    END IF
END FOR

Sort eligible students by:
    1. Higher aggregate marks
    2. Higher attendance
    3. Lower family income

IF eligible_students is empty
    Display "No students are eligible"
ELSE
    Display ranked eligible students
END IF

END
```

7. Python Source Code

```
# University Scholarship Allocation

students = [
    ["Ravi", 85, [80, 85, 90], 90, 150000],
    ["Sita", 90, [90, 88, 92], 95, 120000],
    ["Raju", 90, [85, 90, 90], 90, 100000],
    ["Priya", 70, [65, 75, 70], 80, 200000],
    ["Kiran", 88, [90, 85, 88], 85, 300000]
]

min_aggregate = 75
min_subject_marks = 60
min_attendance = 75
max_income = 200000

eligible_students = []

for student in students:
    aggregate = student[1]
    subject_marks = student[2]
    attendance = student[3]
    income = student[4]
```

```
    subjects_passed = True

    for marks in subject_marks:
        if marks < min_subject_marks:
            subjects_passed = False

    if (aggregate >= min_aggregate and
        subjects_passed and
        attendance >= min_attendance and
        income <= max_income):
        eligible_students.append(student)

# Rank eligible students
eligible_students.sort(
    key=lambda s: (-s[1], -s[3], s[4])
)

print("UNIVERSITY SCHOLARSHIP ALLOCATION")
print("---------------------------------")

if len(eligible_students) == 0:
    print("No students are eligible.")
else:
    print("Eligible Students Ranked by Merit:\n")

    for rank, student in enumerate(
```

```
    eligible_students, start=1
):
    print("Rank:", rank)
    print("Name:", student[0])
    print("Aggregate Marks:", student[1])
    print("Subject Marks:", student[2])
    print("Attendance:", student[3], "%")
    print("Family Income: Rs.", student[4])
    print()
```

8. Test Cases

Test case	Input condition	Expected result
1	Ravi: aggregate 85, attendance 90%, income ₹1,50,000	Eligible
2	Sita: aggregate 90, attendance 95%, income ₹1,20,000	Eligible, Rank 1
3	Raju: aggregate 90, attendance 90%, income ₹1,00,000	Eligible, Rank 2
4	Priya: aggregate 70	Not eligible
5	Kiran: income ₹3,00,000	Not eligible

9. Output Screenshot



10. Conclusion

The University Scholarship Allocation program successfully checks student eligibility based on academic performance, subject-wise marks, attendance, and family income. It ranks eligible students using aggregate marks and predefined tie-breaking rules. This program demonstrates the use of Python lists, nested lists, loops, conditional statements, and sorting.

5.Progressive Electricity Billing:

1. Problem Statement

Develop a Python program to calculate household electricity bills for four months using progressive electricity rates. The program should include fixed charges, surcharges, and rebates. It should compare monthly bills, identify unusual increases, and display the month with the highest bill.

2. Objectives

- To calculate electricity charges using different consumption slabs.
- To include fixed charges, surcharges, and rebates.
- To compare electricity bills across months.
- To identify unusual increases in bills.
- To find the month with the highest electricity bill.

3. Assumptions

1. Electricity consumption is given for four months.
2. The first 100 units cost ₹2 per unit.
3. Units from 101 to 200 cost ₹3 per unit.
4. Units above 200 cost ₹5 per unit.
5. The fixed charge is ₹50 per month.
6. A 5% surcharge is applied to the energy charge.
7. A 10% rebate is given when consumption is 100 units or less.
8. A bill increase greater than 50% is considered unusual.

Note: These are sample rates for the assignment, not official electricity tariff rates.

4. Algorithm

1. Start the program.
2. Store the month names and electricity units.
3. Set the fixed charge, surcharge rate, and rebate rate.
4. Repeat for each month.
5. Calculate the energy charge using the applicable consumption slab.
6. Calculate the surcharge.
7. Check whether the consumer is eligible for a rebate.
8. Calculate the final bill and store it.
9. Compare each month's bill with the previous month's bill.
10. Display a warning if the increase is greater than 50%.
11. Find and display the highest bill and its month.
12. Stop the program.

5. Pseudocode

```
BEGIN
    Store month names and electricity units
    Set fixed charge = 50
    Set surcharge rate = 5%
    Set rebate rate = 10%

    FOR each month
```

```
        Calculate energy charge using slabs
        Calculate surcharge

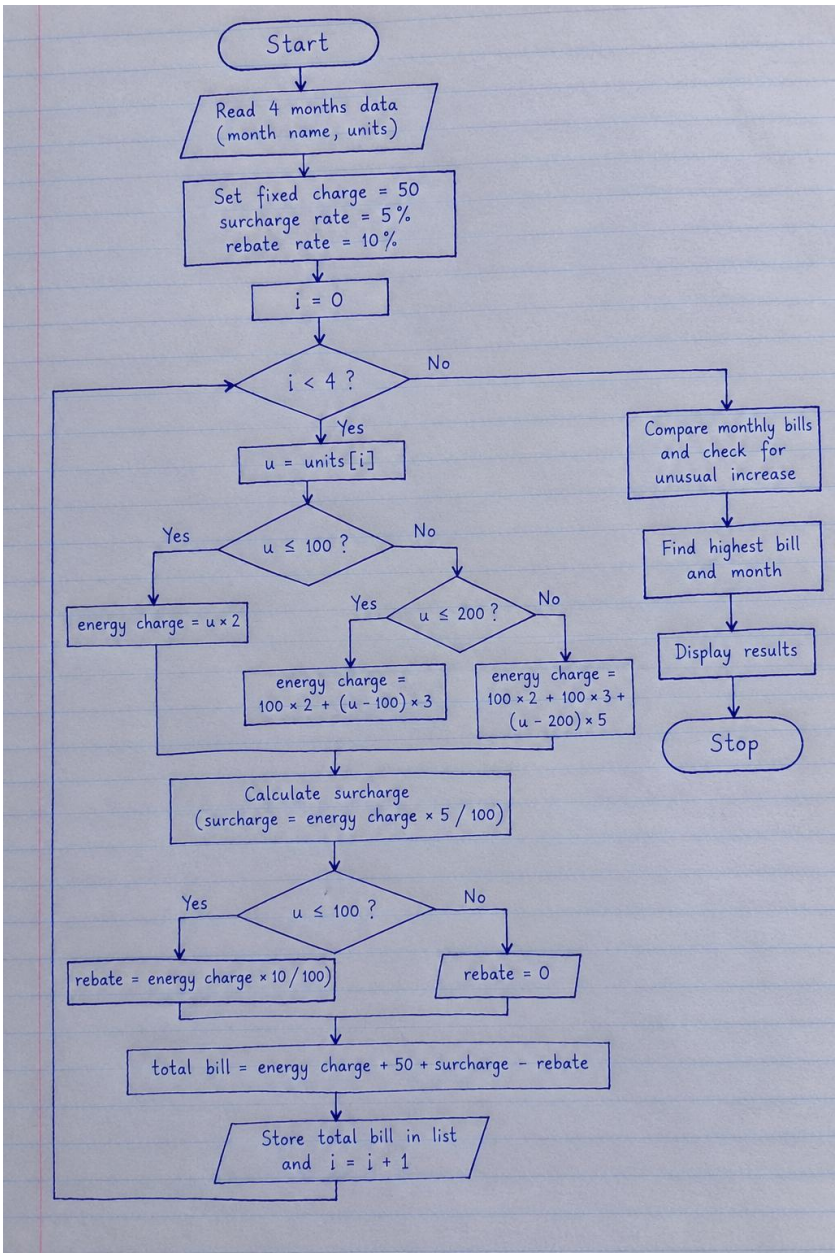
        IF units <= 100
            Calculate rebate
        ELSE
            Set rebate = 0
        END IF

        Calculate total bill
        Store and display bill
    END FOR

    Compare consecutive monthly bills
    IF increase > 50% of previous bill
        Display unusual increase warning
    END IF

    Find and display highest bill and month
END
```

6. Flowchart



7. Python Source Code

Progressive Electricity Billing

Electricity consumption in units for 4 months

months = ["January", "February", "March", "April"] units = [100, 150, 180, 300]

fixed_charge = 50 surcharge_rate = 5 rebate_rate = 10

bills = []

for i in range(len(units)): u = units[i]

```
# Calculate charges using progressive slabs
if u <= 100:
    energy_charge = u * 2
elif u <= 200:
    energy_charge = 100 * 2 + (u - 100) * 3
else:
    energy_charge = 100 * 2 + 100 * 3 + (u - 200) * 5

# Calculate surcharge
surcharge = energy_charge * surcharge_rate / 100

# Calculate rebate for consumption up to 100 units
if u <= 100:
    rebate = energy_charge * rebate_rate / 100
else:
    rebate = 0

# Calculate final bill
total_bill = energy_charge + fixed_charge + surcharge - rebate
bills.append(total_bill)

print("\nMonth:", months[i])
print("Units consumed:", u)
print("Energy charge: Rs.", round(energy_charge, 2))
print("Fixed charge: Rs.", fixed_charge)
print("Surcharge: Rs.", round(surcharge, 2))
print("Rebate: Rs.", round(rebate, 2))
print("Total bill: Rs.", round(total_bill, 2))
```

Compare monthly bills

print("\n--- Monthly Bill Comparison ---")

for i in range(1, len(bills)): increase = bills[i] - bills[i - 1]

```
print(months[i - 1], "to", months[i])
print("Bill increase: Rs.", round(increase, 2))

# Identify unusual increases above 50%
if increase > bills[i - 1] * 0.50:
    print("Warning: Unusual bill increase detected!")
else:
    print("Bill increase is within normal range.")
```

Identify highest bill

highest_bill = max(bills) highest_month = months[bills.index(highest_bill)]

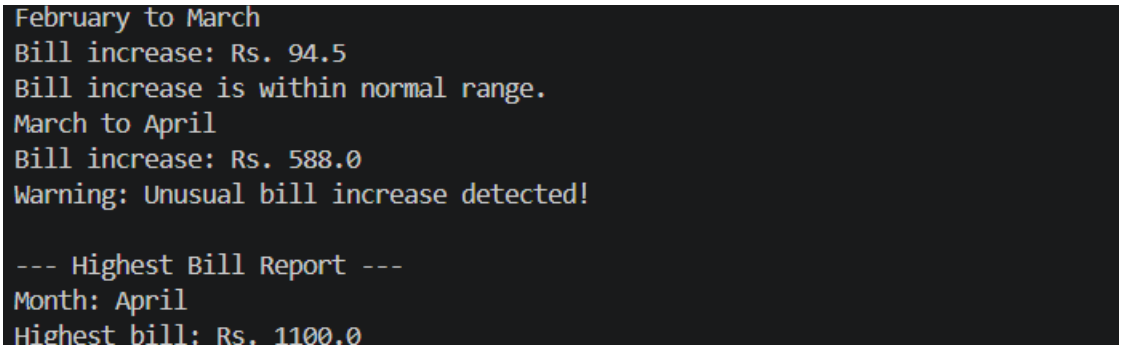
print("\n--- Highest Bill Report ---") print("Month:", highest_month) print("Highest bill: Rs.", round(highest_bill, 2))

8. Test Cases

Test case	Units consumed	Expected energy charge	Rebate
1	50	₹100	₹10
2	100	₹200	₹20
3	150	₹350	₹0
4	180	₹440	₹0
5	300	₹900	₹0

These tests check the different consumption slabs and rebate condition.

9. Output Screenshot



10. Conclusion

The Progressive Electricity Billing program successfully calculates electricity bills using progressive consumption slabs. It includes fixed charges, surcharges, and rebates. It compares bills across months, identifies unusual increases, and displays the month with the highest bill. This program demonstrates Python concepts such as lists, loops, conditional statements, arithmetic operations, and built-in functions.

6.Railway Fare and Reservation:

1. Problem Statement

To develop a Python program to calculate railway fares for multiple passengers based on distance, travel class, age concessions, reservation fees, and cancellation penalties.

2. Objectives

- Calculate fares based on distance.
- Apply AC and Sleeper class charges.
- Provide age-based concessions.
- Calculate cancellation penalties and refunds.

3. Assumptions

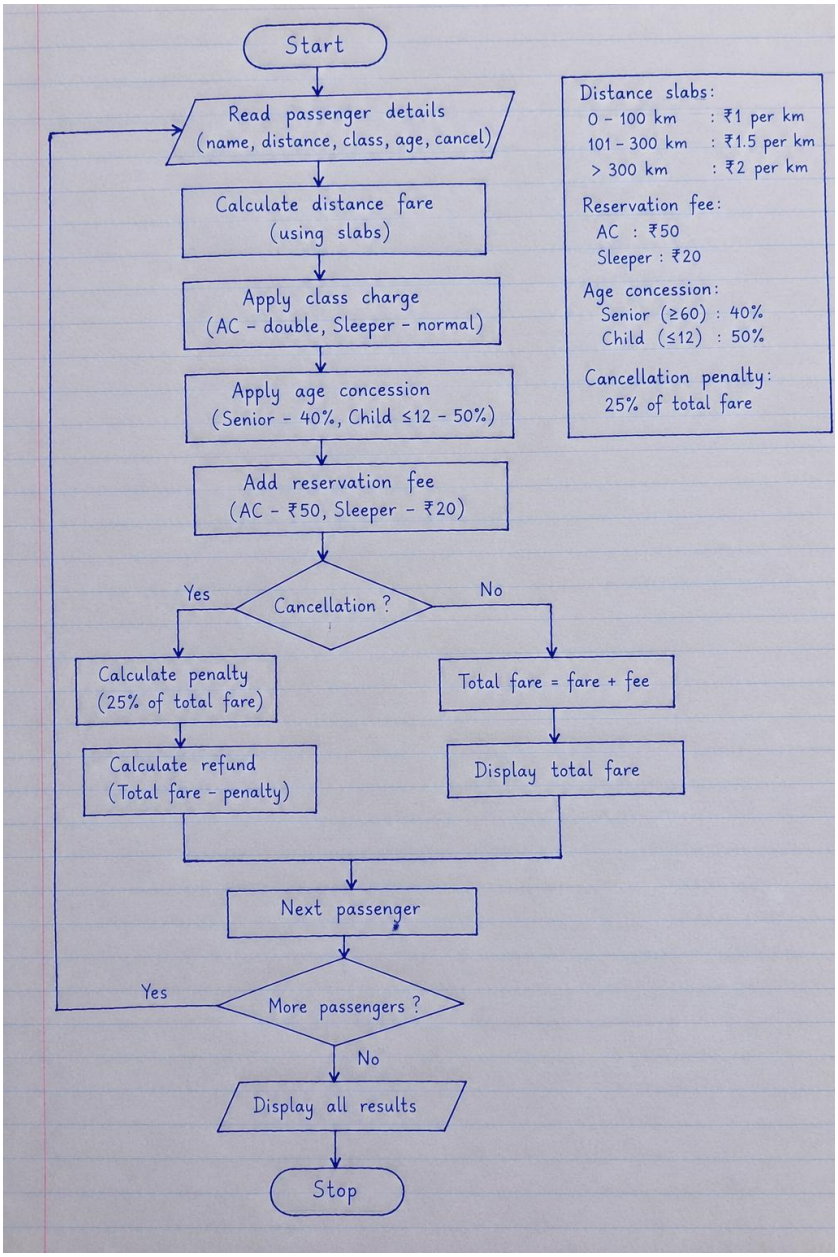
- First 100 km: ₹1 per km.
- 101–300 km: ₹1.50 per additional km.
- Above 300 km: ₹2 per additional km.
- AC fare is double.
- Senior citizens receive 40% concession.
- Children aged 12 or below receive 50% concession.
- Reservation fee: AC ₹50, Sleeper ₹20.
- Cancellation penalty: 25% of total fare.

These are sample rates for the assignment, not official railway fares.

4. Algorithm

1. Start.
2. Read passenger details.
3. Calculate fare based on distance.
4. Apply class charges and age concessions.
5. Add reservation fee.
6. Check cancellation status.
7. Calculate refund and penalty if cancelled.
8. Display the results.
9. Repeat for all passengers.
10. Stop.

5.Flowchart



6. Pseudocode

```
BEGIN
  FOR each passenger
    Calculate distance fare
    Apply class charges
    Apply age concession
```

```
    Add reservation fee

    IF ticket is cancelled
      Calculate penalty and refund
    ELSE
      Display total fare
    END IF
  END FOR
END
```

7. Python Source Code

Railway Fare and Reservation Calculator

passengers = [("Ravi", 100, "Sleeper", 20, "No"), ("Sita", 250, "AC", 65, "No"), ("Arun", 400, "Sleeper", 10, "Yes")]

for name, distance, cls, age, cancel in passengers:

```
# Distance-based fare
if distance <= 100:
    fare = distance * 1
elif distance <= 300:
    fare = 100 + (distance - 100) * 1.5
else:
    fare = 400 + (distance - 300) * 2

if cls == "AC":
    fare *= 2

# Age concession
if age >= 60:
    fare *= 0.60
elif age <= 12:
    fare *= 0.50

# Reservation fee
fee = 50 if cls == "AC" else 20
total = fare + fee
```

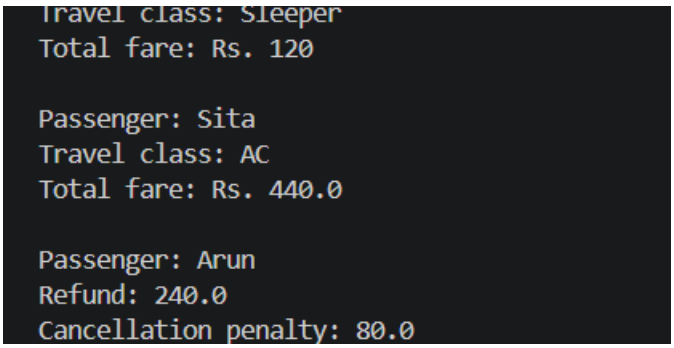
```
print("\nPassenger:", name)

if cancel == "Yes":
    penalty = total * 0.25
    print("Refund:", round(total - penalty, 2))
    print("Cancellation penalty:", round(penalty, 2))
else:
    print("Travel class:", cls)
    print("Total fare: Rs.", round(total, 2))
```

8. Test Cases

Passenger	Distance	Class	Age	Status
Ravi	100 km	Sleeper	20	Booked
Sita	250 km	AC	65	Booked
Arun	400 km	Sleeper	10	Cancelled

9. Output Screenshot



10. Conclusion

The program calculates railway fares for multiple passengers and applies distance slabs, travel class charges, age concessions, reservation fees, and cancellation penalties. It demonstrates Python lists, tuples, loops, conditional statements, and arithmetic operations.

7.Multi - Currency Travel Budget Analysis:

1. Problem Statement

To develop a Python program that converts travel expenses into Indian Rupees using exchange rates, adds conversion fees, compares total expenses with the budget, and identifies the largest expense category.

2. Objectives

- Convert different currencies into INR.
- Calculate currency conversion fees.
- Compare expenses with the planned budget.
- Identify the largest expense category.

3. Assumptions

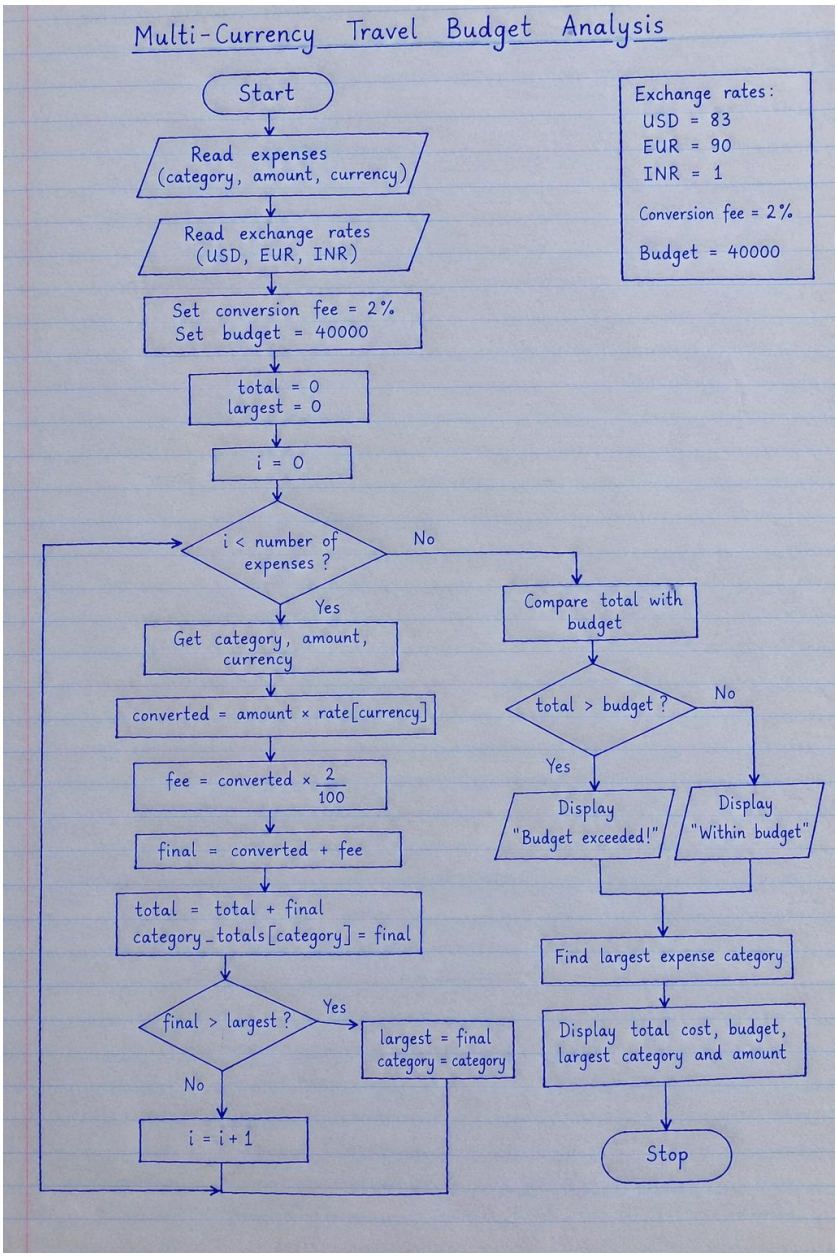
- USD rate: ₹83.
- EUR rate: ₹90.
- INR rate: ₹1.
- Conversion fee: 2%.
- Travel budget: ₹40,000.

Exchange rates are sample values for this assignment.

4. Algorithm

1. Start.
2. Store expenses and exchange rates.
3. Set conversion fee and budget.
4. Convert each expense into INR.
5. Add the conversion fee.
6. Calculate total travel expenses.
7. Compare the total with the budget.
8. Find the largest expense category.
9. Display results.
10. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  Store expenses and exchange rates
  Set conversion fee and budget
  FOR each expense
    Convert amount into INR
```

```
    Calculate conversion fee
    Add fee to expense
    Update total
  END FOR

  Compare total with budget
  Find largest expense category
  Display results
END
```

7. Python Source Code

```
# Multi-Currency Travel Budget Analysis


expenses = [
  ("Hotel", 200, "USD"),
  ("Food", 100, "EUR"),
  ("Shopping", 150, "USD"),
  ("Transport", 5000, "INR")
]


rates = {"USD": 83, "EUR": 90, "INR": 1}
conversion_fee = 2
budget = 40000


total = 0
```

```
category_totals = {}

for category, amount, currency in expenses:
    converted = amount * rates[currency]
    fee = converted * conversion_fee / 100
    final_amount = converted + fee

    category_totals[category] = final_amount
    total += final_amount

    print(category, "Cost in INR:", round(final_amount, 2))

print("\nTotal Travel Cost: Rs.", round(total, 2))
print("Budget: Rs.", budget)

if total > budget:
    print("Warning: Budget exceeded!")
else:
    print("Travel is within budget.")

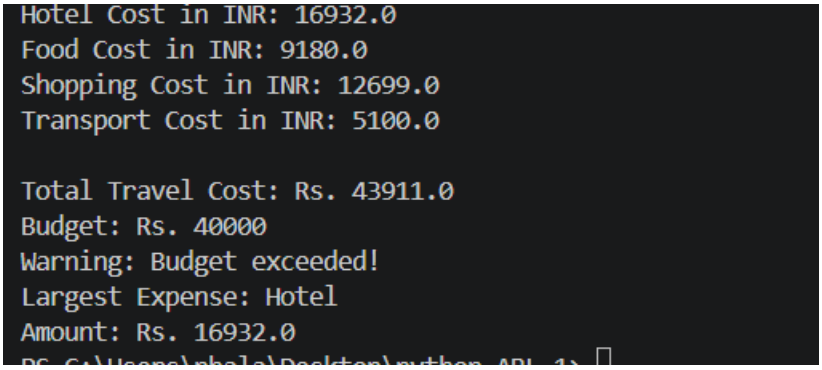
largest = max(category_totals, key=category_totals.get)
print("Largest Expense:", largest)
```

```
print("Amount: Rs.", round(category_totals[largest], 2))
```

8. Test Cases

Category	Amount	Currency	Expected cost in INR
Hotel	200	USD	₹16,932
Food	100	EUR	₹9,180
Shopping	150	USD	₹12,699
Transport	5,000	INR	₹5,100

9. Output Screenshot



10. Conclusion

The program converts travel expenses into INR, calculates conversion fees, checks whether the budget is exceeded, and identifies the largest expense category. It demonstrates Python lists, dictionaries, loops, conditional statements, and built-in functions.

8.Smart Parking Fee Management:

1. Problem Statement

To develop a Python program to calculate parking fees based on vehicle type, entry and exit times, overnight charges, daily limits, and penalties. The program also validates the time inputs.

2. Objectives

- Calculate parking duration.
- Apply charges for different vehicle types.
- Include overnight charges and daily limits.
- Apply penalties for long parking durations.
- Validate time inputs.

3. Assumptions

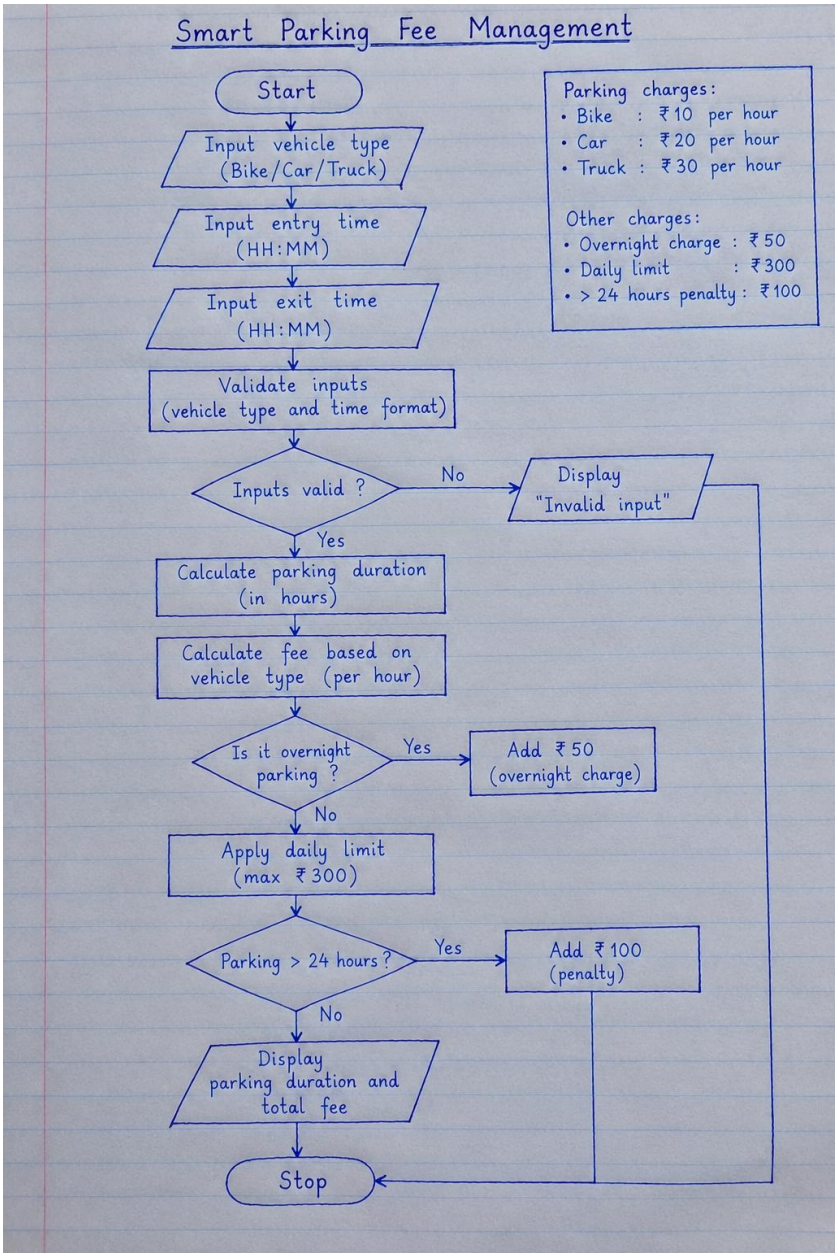
- Bike: ₹10 per hour.
- Car: ₹20 per hour.
- Truck: ₹30 per hour.
- Overnight charge: ₹50.
- Daily fee limit: ₹300.
- Parking beyond 24 hours incurs a ₹100 penalty.

4. Algorithm

1. Start.
2. Enter vehicle type, entry time, and exit time.
3. Validate the vehicle type and time format.
4. Calculate parking duration.
5. Calculate the fee based on vehicle type.

6. Add overnight charges if applicable.
7. Apply the daily fee limit and penalty.
8. Display parking duration and total fee.
9. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  Read vehicle type and entry/exit times
  Validate inputs

  IF inputs are valid
    Calculate parking duration
    Calculate fee based on vehicle type
    Add overnight charge if applicable
    Apply daily limit
    IF duration > 24 hours
      Add penalty
    END IF
    Display duration and fee
  ELSE
    Display error message
  END IF
END
```

7. Python Source Code

Smart Parking Fee Management

vehicle = input("Enter vehicle type (Bike/Car/Truck): ").title() entry = input("Enter entry time (HH:MM): ") exit_time = input("Enter exit time (HH:MM): ")

try: from datetime import datetime, timedelta

```
start = datetime.strptime(entry, "%H:%M")
end = datetime.strptime(exit_time, "%H:%M")

if vehicle not in ["Bike", "Car", "Truck"]:
    print("Invalid vehicle type!")

else:
    if end < start:
        end += timedelta(days=1)
```

```
    minutes = int((end - start).total_seconds() / 60)

    if minutes == 0:
        print("Invalid parking duration!")
    else:
        hours = (minutes + 59) // 60

        rates = {"Bike": 10, "Car": 20, "Truck": 30}
        fee = hours * rates[vehicle]

        # Overnight parking charge
        if end.date() > start.date():
            fee += 50

        # Daily limit
        fee = min(fee, 300)

        # Penalty for parking over 24 hours
        if minutes > 1440:
            fee += 100

        print("Parking duration:", hours, "hours")
        print("Total parking fee: Rs.", fee)
```

except ValueError: print("Invalid time! Enter time in HH:MM format.")

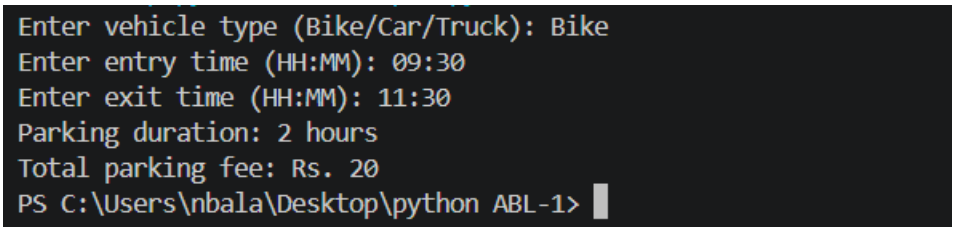
8. Test Cases

Vehicle	Entry	Exit	Expected result
Bike	09:00	11:00	₹20
Car	10:00	13:00	₹60

Truck	08:00	10:00	₹60
Car	22:00	02:00	₹130
Bike	10:00	10:00	Invalid parking duration

These results use the sample fee rules in the program.

9. Output Screenshot



10. Conclusion

The program calculates parking fees according to vehicle type and parking duration. It also handles overnight parking, fee limits, penalties, and invalid time inputs using Python conditional statements, dictionaries, and exception handling.

9.Multi Event Sports Scheduling and Scoring:

1. Problem Statement

To develop a Python program that identifies scheduling conflicts between sports events, displays team scores, and generates final rankings based on points.

2. Objectives

- Represent event schedules using tuples and lists.
- Identify overlapping event timings.
- Display team scores.
- Rank teams according to their points.

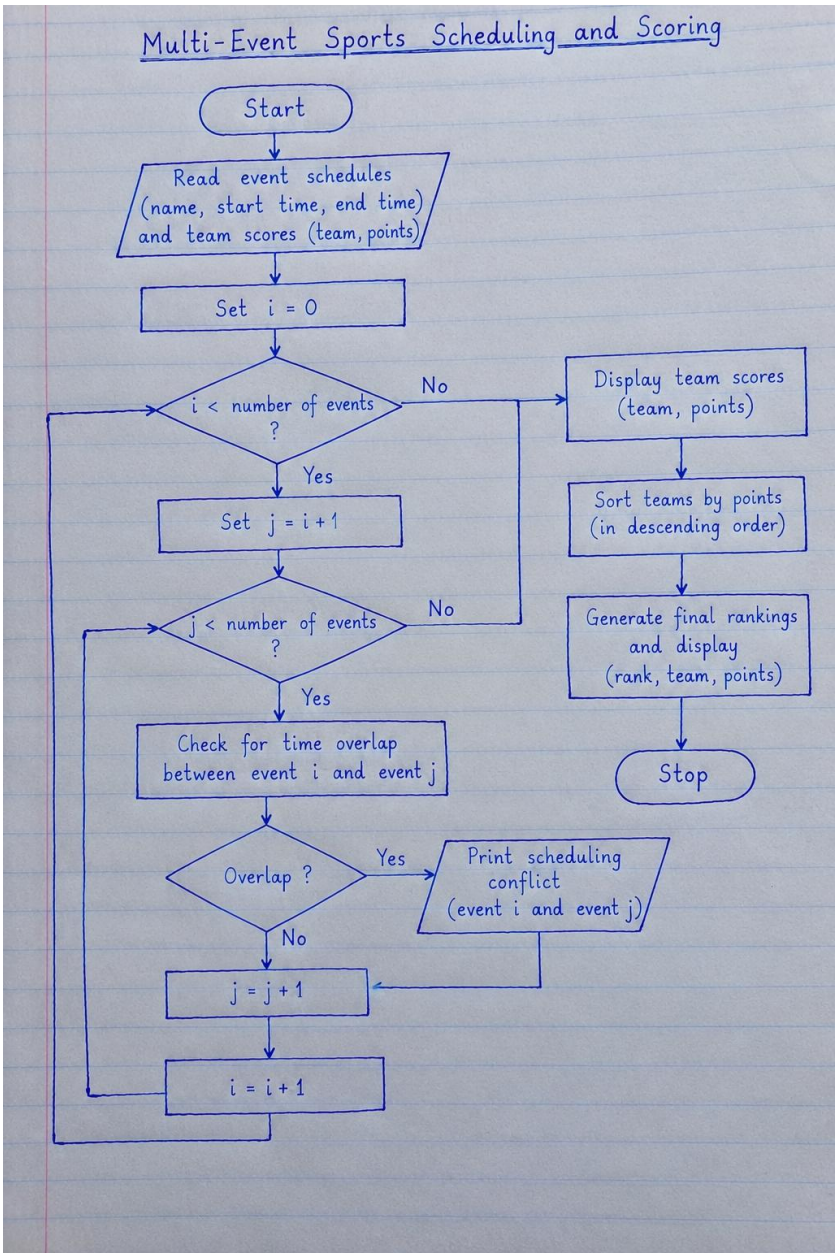
3. Assumptions

- Event times are given in `HH:MM` format.
- Events that overlap have a scheduling conflict.
- Teams with higher points receive higher rankings.
- Team scores are already calculated.

4. Algorithm

1. Start.
2. Store event schedules and team scores.
3. Compare each pair of events.
4. Display any scheduling conflicts.
5. Display team scores.
6. Sort teams in descending order of points.
7. Display final rankings.
8. Stop

5. Flowchart



6. Pseudocode

```
BEGIN
  Store events and team scores

  FOR each pair of events
    IF event times overlap
      Display scheduling conflict
```

```
    END IF
  END FOR

  Display team scores
  Sort teams by points in descending order
  Display final rankings
END
```

7. Python Source Code

Multi-Event Sports Scheduling and Scoring

events = [("Football", "09:00", "10:00"), ("Basketball", "10:00", "11:00"), ("Cricket", "09:30", "10:30")]

scores = [("Team A", 10), ("Team B", 15), ("Team C", 12)]

Check scheduling conflicts

print("Scheduling Conflicts:")

for i in range(len(events)): for j in range(i + 1, len(events)): name1, start1, end1 = events[i] name2, start2, end2 = events[j]

```
   if start1 < end2 and start2 < end1:
        print(name1, "and", name2, "have a conflict")
```

Display team scores

print("\nTeam Scores:") for team, points in scores:

print(team, ":", points, "points")

Generate final rankings

scores.sort(key=lambda x: x[1], reverse=True)

print("\nFinal Rankings:")

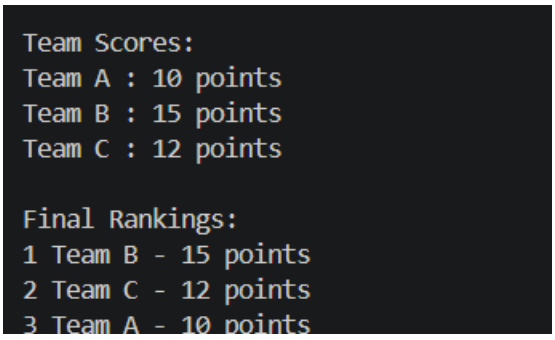
```
 for rank, (team, points) in enumerate(scores, start=1):

print(rank, team, "-", points, "points")
```

8. Test Cases

Test case	Input	Expected result
1	Football: 09:00–10:00; Cricket: 09:30–10:30	Conflict detected
2	Football: 09:00–10:00; Basketball: 10:00–11:00	No conflict
3	Team A: 10 points; Team B: 15 points	Team B ranks first
4	Team C: 12 points	Team C ranks second

9. Output Screenshot



10. Conclusion

The program identifies overlapping sports events and ranks teams according to their points. It demonstrates Python lists, tuples, nested loops, conditional statements, and sorting.

10.University Hostel Room Allocation:

1. Problem Statement

To develop a Python program to allocate students to hostel rooms based on eligibility and room capacity, detect duplicate assignments, and calculate room occupancy.

2. Objectives

- Allocate students to eligible rooms.
- Ensure room capacity is not exceeded.
- Detect duplicate student assignments.
- Calculate occupied and unallocated student counts.

3. Assumptions

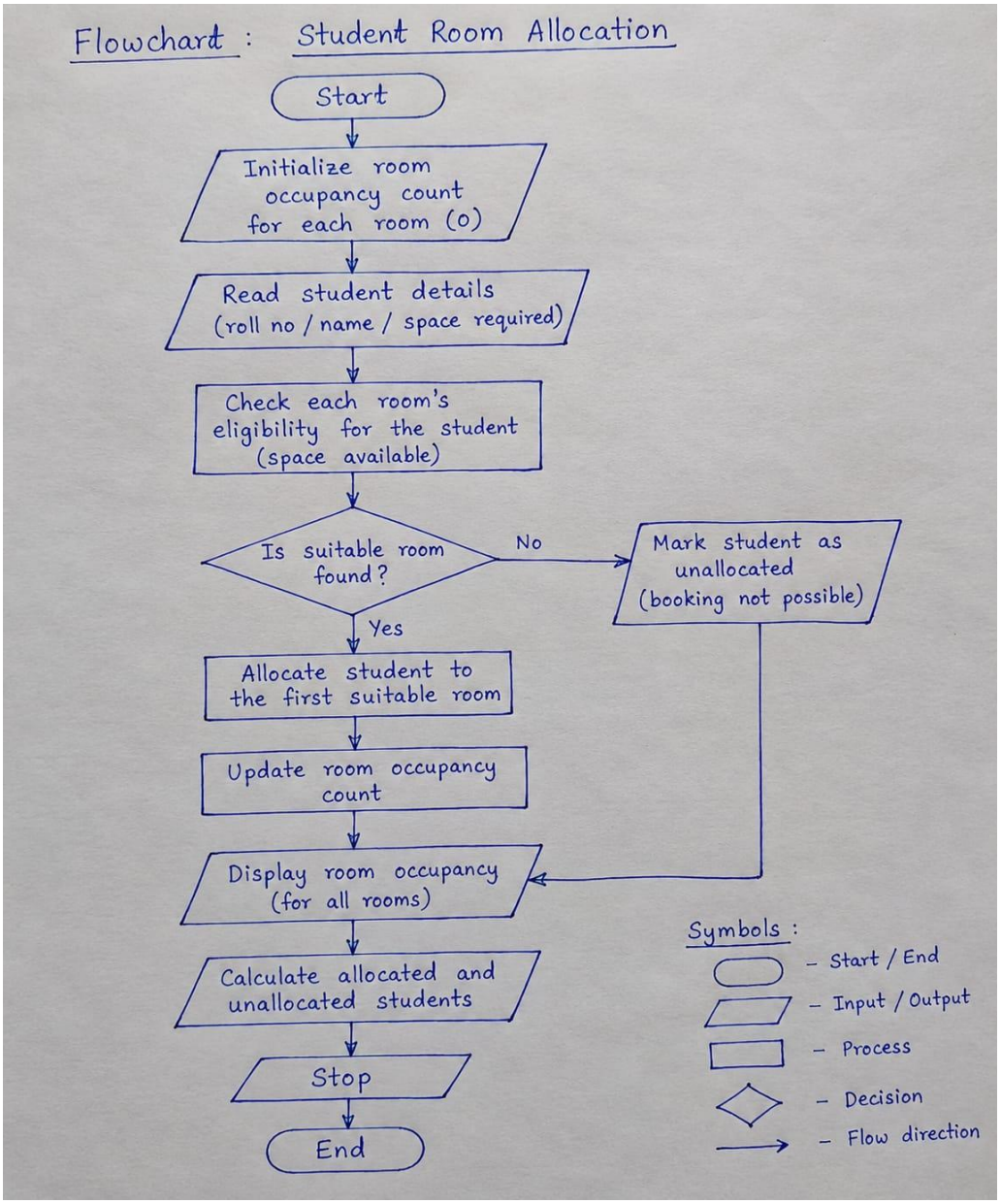
- Each student has a gender category.
- Rooms have a fixed capacity and eligibility category.
- Each student can occupy only one room.

4. Algorithm

1. Start.
2. Store student and room details.
3. Initialize room occupancy counts.
4. Check each student's eligibility for each room.
5. Allocate eligible students if space is available.
6. Check for duplicate assignments.

7. Display room occupancy.
8. Calculate allocated and unallocated students.
9. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  Store students and rooms
  Initialize room occupancy

  FOR each room
    FOR each student
      IF student is eligible AND room has space
        IF student is not already allocated
          Allocate student
        END IF
      END IF
    END FOR
  END FOR

  Check duplicate assignments
  Display room occupancy
  Calculate allocated and unallocated students
END
```

7. Python Source Code

University Hostel Room Allocation

students = [("Ravi", "Male"), ("Sita", "Female"), ("Arun", "Male"), ("Priya", "Female")]

rooms = [("Room1", "Male", 2), ("Room2", "Female", 2)]

allocations = [ ]

counts = { }

for room, gender, capacity in rooms: counts[room] = 0

```
for name, student_gender in students:
    if student_gender == gender and counts[room] < capacity:
        if name not in [a[0] for a in allocations]:
            allocations.append((name, room))
```

```
            counts[room] += 1
            print(name, "allocated to", room)
```

Check duplicate assignments

print("\nDuplicate Assignment Check:") names = [a[0] for a in allocations]

if len(names) == len(set(names)):

print("No duplicate assignments.")

else:

print("Duplicate assignments found!")

Display room occupancy

print("\nRoom Occupancy:") for room, gender, capacity in rooms: occupied = counts[room]

print(room, ":", occupied, "/", capacity)

print("\nTotal students allocated:", len(allocations))

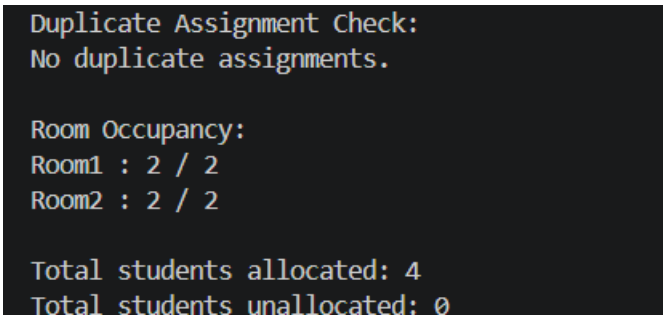
print("Total students unallocated:", len(students) - len(allocations))

8. Test Cases

Test case	Input condition	Expected result
1	Four eligible students and four available beds	All four allocated
2	Room capacity is full	No additional student allocated

3	Student already assigned to a room	Duplicate prevented
4	Student has no eligible room	Student remains unallocated

9. Output Screenshot



10. Conclusion

The program allocates students to eligible hostel rooms according to room capacity. It checks duplicate assignments and calculates room occupancy, allocated students, and unallocated students using Python lists, tuples, loops, dictionaries, and conditional statements.